A Feature-based Analysis of Pivot Rules for the Revised Simplex Method

Rakeeb Hossain [1] **Aristotelis Chaniotis** [1] **Parsa Salimi** [1]

Abstract

This paper investigates various common *pivot rules* for the REVISED SIMPLEX METHOD in solving linear programs (LPs). This is presented from a feature-engineering perspective by first formulating a feature set for quantifiably characterizing LP problem instances. We then vary these parameters on randomized LP instances to determine how pivot rules perform on certain classes of problems. As an extension of our results, we explore the hypothesis that there is no "dominant" pivot rule and that dynamic rule selection using features can lead to faster solves on a per-instance basis. Inspired by SATzilla's similar hypothesis in 2007, we present **FolioPlex**, an online rule-selection strategy for the simplex method that uses our LP feature set to construct an algorithm portfolio of *hardness models*. Finally, we suggest several future directions in applying feature-based machine learning to LP solvers.

1. Introduction and background

The Dantzig's simplex method (Dantzig, 1965) is perhaps the most well known algorithm for solving linear programs. While having a worst-case exponential time complexity (Schrijver, 1989) and thus being theoretically less efficient than some other methods for solving linear programs, such as Khachiyan's polynomial time ellipsoid method, it performs quite well in the average-case (Smale, 1983). Thus, in most practical applications, the simplex algorithm is very efficient. The simplex algorithm also has very good stability, and it is generally much more flexible than other methods (Bixby, 2016). Finally, the simplex method has been the subject of numerous studies, which have, accumulatively, improved it's efficiency by at least four orders of magnitude (Bixby, 2012).

[*]Equal contribution [1]Department of Combinatorics and Optimization, University of Waterloo, Waterloo ON, Canada. Correspondence to: Parsa Salimi <p2salimi@uwaterloo.ca>.

1.1. Some terminology and notation

A *linear program (LP)* is said to be in *standard form* if it is of the following form:

$$\begin{aligned} &\text{minimize } c^\intercal x \\ &\text{subject to } Ax = b \\ &x \geq 0. \end{aligned} \tag{1}$$

We always assume that our linear programs are in standard form. In what follows in this paper unless stated otherwise we assume that $c \in \mathbb{R}^m$, $b \in \mathbb{R}^n$ and $A \in \mathbb{R}^{n \times m}$, m and n are positive integers. The function $c^\intercal x$ is called the *objective function*, the equations defining $Ax = b$ are called the *constraints* and the conditions $x \geq 0$ are the *nonnegativity conditions*.

We will also need the following terminology which is associate with the simplex algorithm. Let B be an ordered set of m points from $\{1, \ldots, m\}$. If A_B is invertible, then B is called a *basis* for (1), and the corresponding variables are called the *basic variables*. The set of other indices $\{1, \ldots, m\} \setminus B$ is denoted by N and referred to as the *nonbasic* set of indices. The corresponding variables x_B are called the *nonbasic variables*. The *basic solution* determined by B is the solution obtained by setting $x_N = 0$ and $x_B = B^{-1}b$. This solution is *primal feasible* if $B^{-1}b \geq 0$. The vectors $D_N := c_N - A_N^\intercal B^{-1\intercal} c_B$ are called the *reduced costs* of the objective function with respect to the nonbasic variables. For a more detailed description of the terminology which is described above we refer the interested reader to (Gärtner & Matoušekernd, 2007).

1.2. The Revised Simplex Method

Broadly speaking, there are two different variations of the simplex method. The first is the algorithm first proposed by Dantzig which is referred as the PRIMAL SIMPLEX ALGORITHM. The other variant is the Dual simplex algorithm, first proposed by (Lemke, 1954). Generally speaking, the DUAL SIMPLEX ALGORITHM is the preferred method, for several reasons. It is easier to achieve a dual feasible solution (Hall, 2020). It is generally faster, and works especially well in context of mixed integer programming (Bixby, 2016). Moreover, dual variants of the steepest edge pivot

rule (which will be discussed later in the primal setting), generally work much better compared to the primal version (Forrest & Goldfarb, 1992). For a detailed description of the DUAL SIMPLEX ALGORITHM we refer the interested reader in Chapter 8 of (Paris, 2016). In this paper we focus on the PRIMAL SIMPLEX ALGORITHM, and in particular in the efficient implementation of this algorithm which is known as the REVISED SIMPLEX METHOD (Chvatal et al., 1983).

In short the REVISED SIMPLEX METHOD is an iterative algorithm, which initiating with a feasible basis, either detects that the given LP is unbounded or at each iteration moves from a given feasible basis B to another feasible basis, say B', which corresponds to an LP's feasible solution with "better" objective value than the objective value of the feasible solution corresponding to B, and after a finite number of iterations the algorithm ends up with a feasible basis which determine an optimal solution for the given LP problem (except the cases of Degeneracy and Cycling which we can handle as described below).

Below we appose a high-level description of an iteration REVISED SIMPLEX METHOD, the reader interested in details is refereed to Chapter 7 of (Chvatal et al., 1983).

Algorithm 1 REVISED SIMPLEX METHOD

INPUT: A feasible basis B, vectors $x_B = A^{-1}b$ and the reduced costs $D_N := c_N - A_N^\intercal B^{-1\intercal} c_B$

STEP 1 (PRICING):

if $D_N \geq 0$ **then**
  STOP.The current feasible basis determine an optimal solution for the LP.
else
  set $j =$ `pivot(N)`

STEP 2 (FORWARD TRANSFORMATION):
Solve $A_B y = A_j$

STEP 3 (RATIO TEST):

if $y \leq 0$ **then**
  The LP is unbounded,STOP
else
  set $i = argmin\{x_{Bk}/y_k : y_k > 0\}$.

STEP 4 (BACKWARD TRANSFORMATION):
Solve $A_B^\intercal z = e_i$.

STEP 5 (UPDATE): Compute $\alpha_N = -A_N^\intercal z$. Let $B_i = j$. Update x_B using y and update D_N using α_N.

1.3. The pricing step in the REVISED SIMPLEX METHOD

The variables j and i in the above description of the REVISED SIMPLEX METHOD, are called the *entering variable* and the *leaving variable* respectively. The choice of the entering -to the basis- variable at each iteration of the the REVISED SIMPLEX METHOD, when there are several possibilities, is of particular importance, since it affects significantly the total number of iterations and thus the overall run-time of the algorithm. In this paper we are concerned with the choice of the function `pivot` in the pricing step. We follow the relevant literature and call such functions *pivot rules*. We remark that in case of a tie in the Step 3 (ratio test), we also need a *tie-breaking* rule, but this choice is of less importance, for the overall algorithm's run-time, than specifying the entering variable. In Section 2 we analyze the pivot rules which we examine in this paper and several other aspects of the REVISED SIMPLEX METHOD related with the pivot rules.

1.4. Algorithm portfolios

We briefly introduce the concept of algorithm portfolios which will be useful when discussing FolioPlex. The concept of "algorithm portfolios" was originally coined for the general strategy of running several algorithms in parallel, with varying amounts of CPU time assigned to each (Huberman et al., 1997). This has the advantage of "better" algorithms performance-wise terminating earlier in the computation when a result is found. Algorithm portfolios have, however, evolved more generally to represent selecting some subset of black-box algorithms to solve a problem, either sequentially or in parallel. There has been notation developed for this, such as an (a, b)-of-n portfolio referring to a strategy that select at least a and at most b strategies out of a portfolio of size n to solve a problem. Regardless of the specifications, each application of algorithm portfolios operate on a similar assumption for the problem they are solving - there is no "best" or "winner-take-all" algorithm for the problem. Put differently, algorithm portfolios attempt to "exploit lack of correlation in the best-case performance of several algorithms in order to obtain improved performance in the average case" (Xu et al., 2008).

In this paper, we focus only on sequential $(1, 1)$-of-n portfolios. That is, to solve a problem, we select exactly one algorithm from a portfolio to be run by itself.

2. Pivot Rules

In this section we present the pivot rules which we examine in this paper and we also discuss some theoretical aspects of these pivot rules. In particular, the four pivot rules which we examine are the following:

DANTZIG'S OR THE LARGEST COEFFICIENT RULE:
This is the original pivot rule suggested by George Dantzig (Dantzig, 1965) and thus the first pivoting rule that was used in the simplex algorithm. This rule selects an entering variable which has the largest coefficient in the row of the objective function.

BLAND'S RULE:
Robert G. Bland proposed in 1977 (Bland, 1977) a pivot rule which selects as an entering variable the first among the available ones, that is, the variable with the smallest index. This pivoting rule, also specifies the leaving variable (in case that there are several possibilities) by choosing again the one with the smallest index. We remark that BLAND'S RULE is of great theoretical importance since the simplex method with this pivoting rule avoids cycling. Cycling is the only possibility in which the simplex method fails (Gärtner & Matoušekernd, 2007).

STEEPEST EDGE RULE:
According to this pivot rule (Harris, 1973) as an entering variable is chosen a variable whose entering into the basis moves the current basic feasible solution in a direction closest to the direction of the vector c. Thus, the Steepest Edge Rule selects as an entering variable the variable with the most objective value reduction per unit distance.

RANDOM EDGE RULE:
This is the simplest example of a randomized pivot rule, where the choice of the entering variable uses random numbers in some way. This rule selects the entering variable uniformly at random, among all possible entering variables.

We remark that the REVISED SIMPLEX METHOD with any of the aforementioned pivot rules has been proved to have an exponential worst-case time-complexity (see e.g. (Klee & Minty, 1972), (Jeroslow, 1973), (Avis & Chvátal, 1978), (Goldfarb & Sit, 1979), (Matoušek & Szabó, 2006)).
It is also noteworthy that despite the fact that the REVISED SIMPLEX METHOD with the RANDOM EDGE RULE pivot rule has an exponential worst-case time-complexity, it has been proved that it has polynomial expected time-complexity (Gärtner & Kaibel, 2007). More generally randomized pivot rules are of great theoretical interest since the best bounds on the worst case run-time of the simplex algorithm have been proved for such pivots (see e.g.(Kalai, 1992), (Friedmann et al., 2011)).

In order to stress the important affect of the choice of the pivot rules on the number of iterations and the overall run-time of the REVISED SIMPLEX METHOD we also refer the interested reader to the computational results of the experiments of (Ploskas, 2016) on large LPs.

3. A Feature Set for Characterizing LP Instances

In order to investigate how pivot rules perform on certain LP instances, we require a way to classify problem instances. One method to do this, which is heavily used by the SAT solving community, is to group problems by their application category. For example, the annual SAT Competition has featured problem categories ranging from hard combinatorial to software verification instances. However, what differs between categories are structural differences in the LP which are captured within the matrix A and vectors b, c. Therefore, we can formulate a quantifiable feature-based characterization of LPs that capture these structural differences. We propose several features and categories of features in this section.

Coefficient statistics: These are statistics of the entries of each of A, b, c. To name a few, consider the minimum, maximum, mean, variation coefficient, and entropy of each entry a_{ij} in A.

Constraint graph statistics: An LP can be modelled as a bipartite variable constraint graph G (Bowly et al., 2020). The vertices of G represent the variables x_i and the constraints u_j. We define G as follows: $x_i u_j \in E(G) \iff A_{ij} \neq 0$. Thus, the variables and constraints form a bipartition of G. We can extract several features from this graph such as the mean, variation, min, max, and entropy of the degree of variable and constraint vertices. For variable vertices, this measures statistics on the number of non-zero elements in each column of A and, for constraint vertices, the number of non-zero elements in each row of A. We can easily extract other graphical properties from the feature-based graph machine-learning literature, but for simplicity only propose this one.

Structural properties: These features investigate algebraic properties of A, b, c. We include the dimensions of A, which correspond to the number of variables and constraints. We also include the 2-norm of b and c and the *operator norm* of A:

$$||A||_2 = \sup\{\frac{||Ax||_2}{||x||_2} : x \in \mathbb{R}^n, ||x||_2 = 1\}$$

We include the *condition number* with respect to L^2, which is frequently used in statistics as a measure of multicollinearity: $\frac{\sigma_{max}(A)}{\sigma_{min}(A)}$ where $\sigma_{max}(A)$ and $\sigma_{min}(A)$ denote maximal and minimal singular values of A, respectively. Finally, we include the boolean feature *degeneracy* which asks whether or not there exists a BFS B such that $\exists i \in B, x_i = 0$. Degeneracy is an important feature since many pivot rules degrade in performance (and even cycle) when run on degenerate LPs.

Sparsity: The sparsity of a matrix $A \in R^{n \times m}$ measures

Coefficient statistics:
A, b, c entry statistics
Row-normalized statistics: $\{\frac{a_{ij}}{b_i} \mid b_i \neq 0\}$
Column-normalized statistics: $\{\frac{a_{ij}}{c_j} \mid c_j \neq 0\}$
Constraint graph statistics
Variable node statistics
Constraint node statistics
Constraint degree-normalized b statistics: $\{\frac{b_i}{\delta(u_i)}\}$
Variable degree-normalized c statistics: $\{\frac{c_i}{\delta(x_i)}\}$
Structural properties
Sparsity of A
Degeneracy of LP specified by A, b, c
Dimensions n, m of A
2-norm of b and c
Operator norm of A using 2-norm
Condition number of A wrt L^2

Table 1. Proposed feature set for characterizing LPs. Here, "statistics" refers to mean, min, max, variation, and entropy of a quantity.

the proportion of the number of zero elements in A. This is also a structural property, but is written separately for emphasis. In practice, this is an important metric as it measures the average number of variables in a constraint and, intuitively, the average complexity of a constraint. Formally, the sparsity $s(A)$ is defined as:

$$s(A) = \frac{\sum_{i=1}^{n} \sum_{j=1}^{n} I\{a_{ij} = 0\}}{n \times m}$$

A summary of our proposed feature set is presented in Table 1. Implementations of most of these feature computations are available at github.com/rakeeb123/simplex-study.

4. Evaluation of Pivot Rules' Performance

4.1. Experimental setup

We used an open source Python/Numpy implementation of the revised simplex algorithm: github.com/hasan-kamal/Linear-Program-Solvers. We added BLAND'S RULE, DANTZIG'S RULE, STEEPEST EDGE RULE, and RANDOM EDGE RULE as a randomized baseline.

We implemented methods for generating random LPs that allow controlling for sparsity, dimension, entry value range, and degeneracy. To ensure that we are generating a feasible LP, we generate a random vector $x_{rand} \in R^m$ and set $b = Ax_{rand}$ during the generation. We also collected solver statistics, including average time spent on pivoting and number of pivot steps.

Experiments were run on a Compute Canada `cedar` cluster using an Intel E5-2683 v4 Broadwell @ 2.1Ghz CPU.

4.2. Results

The following results were performed by varying 3 parameters: pivot rule, number of constraints in LP, and sparsity. The number of variables was fixed to $m = 75$ across each experiment. The range of values for each entry in the LP were selected uniformly at random from $[-100, 100]$. We present results for Steepest Edge, Dantzig's rule, Bland's rule, and Random Edge.

s/n	0.1	0.3	0.5	0.7	0.9
25	58.2	56.8	50.2	49.4	38.4
50	98.4	106.8	95.6	104.2	93.8
75	99.2	101.2	106.4	108.2	95.6

Table 2. Average number of iterations for Steepest Edge Rule after varying sparsity and number of constraints. Average is taken across 15 random LPs.

s/n	0.1	0.3	0.5	0.7	0.9
25	56.2	51.0	53.6	53.2	56.2
50	110.2	113.8	109.8	107.2	102.2
75	96.8	98.6	100.2	102.2	101.8

Table 3. Average number of iterations for Dantzig's Rule after varying sparsity and number of constraints. Average is taken across 15 random LPs.

s/n	0.1	0.3	0.5	0.7	0.9
25	164.3	168.9	167.8	152.9	221.8
50	349.75	350.7	341.3	344.2	297.05
75	261.2	247.1	255.4	266.8	205.1

Table 4. Average number of iterations for Bland's Rule after varying sparsity and number of constraints. Average is taken across 15 random LPs.

s/n	0.1	0.3	0.5	0.7	0.9
25	147.4	135.2	128.8	116.4	53.6
50	282.4	296.4	289.0	298.8	186.0
75	369.0	380.8	410.8	408.0	323.0

Table 5. Average number of iterations for Random Edge Rule after varying sparsity and number of constraints. Average is taken across 15 random LPs.

We do not present any data on the actual runtimes of each pivot rule because our implementations of each of the rules were not optimized. Thus, this penalizes more elaborate rules, like Steepest Edge, which are implemented naively.

4.3. Interpretation of the results

From the above results, we notice that varying simple features has a non-trivial impact on the number of pivot iterations taken. We present several observations from the above data:

- Steepest Edge consistently performs the best on more sparse LPs, while Dantzig's performs close to or worse than Random on these LPs. Since most linear programs encountered in practice are very sparse, it appears that an efficient steepest edge rule should be preferred to Dantzig's rule. In fact, the steepest edge rule is used in most current solvers, especially for the dual simplex algorithm (Bixby, 2016).
- Dantzig's performs very well on dense matrices, while Steepest Edge does not
- Dantzig's performance does not significantly fluctuate between square and non-square constraint matrices, while Steepest Edge performs much better on "less square" matrices.
- Bland's rule, while theoretically significant, does not perform any better than the random edge rule. This is plausible given that neither of these pivot rules take into account intrinsic properties of the LPs. Moreover, cycling is a very rare occurrence in practice, and so most simplex implementations ignore the possibility of cycling (Gärtner & Matoušekernd, 2007).

From the above data and brief analysis, we have evidence to support the hypothesis that different classes of LPs (as defined by our proposed features) affect the performance of a rule. That is, there is a relationship between rule performance and the feature set of an LP. This suggests that we may be able to predict how well each pivot rule does using machine-learning techniques. This would be useful since we observed that there was no rule that did the best on every instance. In fact, this is what we present next.

5. FolioPlex: Online Portfolio-based Rule-selection

5.1. Overview

While varying just the metrics of sparsity and matrix size, we observed a significant difference in performance of the pivot rules. We hypothesize that as we vary more complex features (such as those in Table 1) and add a larger set of competitive pivot rules, we would observe more such variation. That is, there is no "dominant" pivot rule, but we can predict which one might perform best based on these features. Therefore, we propose **FolioPlex**, an online pivot rule selection strategy. In summary, we leverage *hardness models* and build an algorithm portfolio to select rules from at solve-time.

5.2. Empirical hardness models

For building algorithm portfolios, (Leyton-Brown et al., 2002) introduced *empirical hardness models*, a statistical model for determining the runtime of an algorithm A on a distribution of problems D. We construct a hardness model for each rule R to form our portfolio as follows:

1. Select a set of features that characterize LPs (or a specific distribution of LPs) (Table 1)
2. Remove extraneous features and refine feature set using *feature selection*
3. Select a training set T and compute a feature vector $\overline{x}_i$ for each problem in T
4. Compute the performance r_i for each problem by the simplex method while using rule R
5. Fit a function $f(\overline{x}_i)$ that maps a feature set $\overline{x}_i$ to a prediction of $\log r_i$ (done via *ridge regression*). We predict $\log r_i$ (as opposed to just r_i) for stability of the linear model.

The above process is performed offline on a training dataset. The result of the process is a function f_i for each pivot rule – this is our hardness model. Thus, we have outlined constructing our portfolio using hardness models.

5.3. Feature selection

In step 2 above, we attempt to refine our manually-selected feature set from Table 1. The reason this is necessary is because it is well known that highly correlated and, hence, redundant features deteriorate the performance of predictive models by introducing multicollinearity (Read & Belsley, 1991). That is, small perturbations in the input lead to large variations in the output when correlated features are present. The same also goes for uninformative features. Therefore, we would like to refine our feature set as much as possible. We use *forward selection*, an iterative procedure that greedily adds the feature that reduced cross-validation error the most and terminates when no such statistically significant feature exists, as described in Algorithm 2.

Before performing *forward selection*, we can also attempt to augment the modelling ability of our features. Since regression models express the output (the runtime) as a linear function of the inputs, it is difficult to capture non-linear relationships. Therefore, we perform *quadratic basis function expansion* before performing forward selection (Xu et al., 2008). That is, we add pairs of products of each feature $x_i x_j$ to our feature set.

Algorithm 2 Forward selection

Input: Set of features $\overline{x}$
Initialize empty set of features F;
$c_{prev} = \infty$;
while $F \neq \overline{x}$, do:
  for each $x_i \in \overline{x} \setminus F$:
    train regression model f_i using feature set $F \cup \{x_i\}$;
    get cross-validation error c_i of f;
  $c_{min} = \min\{\text{each } c_i\}$;
  if $c_{prev} - c_{min} < \epsilon$:
    break;
  $c_{prev} = c_{min}$;
return F;

Overall, we obtain a modified feature set $\phi(\overline{x}_i) = \{\phi_1(\overline{x}_i), \ldots, \phi_k(\overline{x}_i)\}$ where k is the length of our new feature set.

5.4. Ridge regression

We note in step 5 that we use ridge regression for our hardness model. This is because we would like to optimize for runtime performance of FolioPlex in order to be competitive, so overhead of running the FolioPlex algorithm should be minimized. For a feature vector $\overline{x}_i$, our hardness model for predicting runtime is just $f(\overline{x}_i) = w^\top \phi(\overline{x}_i)$ for a weight vector w (Xu et al., 2008). This is inexpensive to compute compared to other regression techniques, like lasso and SVM regression. Additionally, ridge regression is practically effective at not overfitting by penalizing models with large coefficients while just as inexpensive as an ordinary least squares model (Wieringen, 2020).

5.5. FolioPlex

After the offline construction of the FolioPlex portfolio, we can augment a solver by adding a pre-processing step to it. This pre-processing step computes the feature vector of the LP and uses the portfolio to detect which rule has the lowest expected runtime.

We can summarize the algorithm with the following steps:

1. Compute feature values for LP and (optionally) run a *pre-solver* for *dynamic features* (described in Section 5.6.2)
2. If feature computation times out, run backup rule
3. Predict each rule's runtime in portfolio using hardness models
4. Use rule predicted to have lowest runtime

5.6. Ongoing work

This section lays out the work that we plan to do on FolioPlex after this paper submission.

5.6.1. Performance experiments

While we have implemented feature computations, implementing the regression model for the portfolio is currently ongoing. Once this is done, constructing the portfolio is straightforward following the information from the previous section and we can test the performance of FolioPlex using an identical setup to Section 4. We will augment our base solver with FolioPlex as a rule (which then selects another rule) at pre-processing time.

In order to improve FolioPlex's flexibility, we also plan to implement a set of other competitive rules for FolioPlex to select from such as the Devex pivot rule.

Finally, instead of just testing on randomized LPs, we plan to test FolioPlex on application instances. In the first experiment, we would like to train FolioPlex with random matrices (as in Section 4) and test against application instances, and in another experiment both train and test on application instances. We will sample our training and test sets from the following 3 databases:

1. NETLIB (NETLIB, 2010)
2. Miplib 3 (Gleixner et al., 2021)
3. Miplib 2010 (Gleixner et al., 2021)

The Miplib sets contain LP relaxations of challenging integer programs and NETLIB is known to have numerically challenging application instances (Illés & Nagy, 2013).

5.6.2. Dynamic features & pre-solvers

After carrying out performance experiments, we would also like to augment the set of features in Table 1 to include informative features about "solver statistics". That is, we would like to incorporate dynamic features, which require running a *pre-solver* to compute for a fixed timeout. For example, statistics on a basic feasible solution could be included, such as the objective value achieved by the BFS and how many pivot steps were required to find the BFS.

Computing these dynamic features required selecting a pre-solver. We select our pre-solver to use the best rule on average on the test set. Running this pre-solver also has the benefit of solving very easy LP instances quickly during pre-processing as opposed to expending effort computing features and running FolioPlex on these easy instances.

6. Related Work

There is extensive literature on the performance of pivot rules, both from a theorist and practictioner's point of view (Terlaky & Zhang, 1993) (Illés & Nagy, 2013) (Shamir, 1987). However, these surveys and experimental papers consider the performance of pivot rules in mostly fixed settings. For example, the surveyed papers in (Shamir, 1987) fix square constraint matrices on bounded, feasible problems except one. This one experiment varies the dimensions of the constraint matrix, but further properties are not explored. Thus, there is a gap in the literature on parametrizations of LPs and performance of pivot rules according to these parameters. This is the gap we aimed to investigate here.

Additionally, feature characterizations of linear programs seem to have not been studied as extensively as other components of LPs. However, recent work has explored the generation of randomized LPs with controllable properties, which required formulating several such properties (Bowly et al., 2020). We chose to include these properties within our feature set.

While there has not been much work on applying algorithm portfolios to LPs, algorithm portfolios are a very well studied field. Similar to our description of FolioPlex, algorithm portfolios have been applied to several hard combinatorial problems and have proven to be very successful in the SAT solver domain (Xu et al., 2008).

7. Conclusion and Future Work

We have proposed a feature characterization of linear programs, experimentally shown how solver performance varies by rule-selection and as a result of varying features, and proposed FolioPlex, a dynamic rule-selection strategy.

Our result in showing that there is likely no "dominant" pivot rule is intuitive but important and has exciting implications. Particularly, we are excited to see how this can be applied to building specialized variants of simplex solvers tailored for specific industrial instances.

We are also excited by the potential of our feature characterization of LPs. This can serve as a starting point for applying feature-based machine-learning techniques to simplex solvers. As an example, we have presented FolioPlex. Another possible application is a decision-tree classifier to predict the "best" pivot rule from feature values. However, we believe that there is much room for improvement on our proposed featureset, which consists of only static properties. As outlined in Section 5.6, previous work has employed so-called dynamic properties, which require running a presolver for a specified timeout to compute (Xu et al., 2008). These dynamic properties have been shown to provide valuable information to the performance of an algorithm on a specific problem instance, and so would be beneficial to explore adding to our featureset. We plan to explore dynamic features in future work.

While we did not have time to experiment with FolioPlex's performance for this paper, we have laid much of the groundwork and look forward to testing this very soon. As a group, we plan to continue the project in this direction, as outlined in Section 5.6.

8. Acknowledgements

Resources used for this work were provided by Compute Canada. We would also like to thank Dr. William Cook and the rest of the CO 255 staff for providing us with the opportunity and guidance for doing this project in the context of the course "CO 255: Introduction to Optimization (Advanced Level)" of the Department of Combinatorics and Optimization of University of Waterloo.

References

Avis, D. and Chvátal, V. Notes on bland's pivoting rule. In *Polyhedral Combinatorics*, pp. 24–34. Springer, 1978.

Bixby, R. A brief history of linear and mixed-integer programming computation. *Documenta Mathematica*, 2012.

Bixby, R. Solving linear programs: The dual simplex algorithm. CO@Work, 2016.

Bland, R. G. New finite pivoting rules for the simplex method. *Mathematics of operations Research*, 2(2):103–107, 1977.

Bowly, S. et al. Generation techniques for linear programming instances with controllable properties. volume 12, pp. 389–415, 2020.

Chvatal, V., Chvatal, V., et al. *Linear programming*. Macmillan, 1983.

Dantzig, G. B. *Linear programming and extensions*, volume 48. Princeton university press, 1965.

Forrest, J. J. and Goldfarb, D. Steepest-edge simplex algorithms for linear programming. *Mathematical Programming*, 57(1-3):341–374, 1992. doi: 10.1007/bf01581089.

Friedmann, O., Hansen, T. D., and Zwick, U. Subexponential lower bounds for randomized pivoting rules for the simplex algorithm. In *Proceedings of the forty-third annual ACM symposium on Theory of computing*, pp. 283–292, 2011.

Gärtner, B. and Kaibel, V. Two new bounds for the random-edge simplex-algorithm. *SIAM Journal on Discrete Mathematics*, 21(1):178–190, 2007.

Gleixner, A., Hendel, G., Gamrath, G., Achterberg, T., Bastubbe, M., Berthold, T., Christophel, P. M., Jarck, K., Koch, T., Linderoth, J., Lübbecke, M., Mittelmann, H. D., Ozyurt, D., Ralphs, T. K., Salvagnin, D., and Shinano, Y. MIPLIB 2017: Data-Driven Compilation of the 6th Mixed-Integer Programming Library. *Mathematical Programming Computation*, 2021. doi: 10.1007/s12532-020-00194-3. URL https://doi.org/10.1007/s12532-020-00194-3.

Goldfarb, D. and Sit, W. Y. Worst case behavior of the steepest edge simplex method. *Discrete Applied Mathematics*, 1(4):277–285, 1979.

Gärtner, B. and Matoušekernd, J. *Understanding and using linear programming*. Springer, 2007.

Hall, J. High performance computational techniquesfor the simplex method. CO@Work, 2020.

Harris, P. M. Pivot selection methods of the devex lp code. *Mathematical programming*, 5(1):1–28, 1973.

Huberman, B. A., Lukose, R. M., and Hogg, T. An economics approach to hard computational problems. *Science*, 275(5296):51–54, 1997. ISSN 0036-8075.

Illés, T. and Nagy, A. Computational aspects of simplex and mbu-simplex algorithms using different anti-cycling pivot rules. *Optimization*, 07 2013. doi: 10.1080/02331934.2013.811666.

Jeroslow, R. G. The simplex algorithm with the pivot rule of maximizing criterion improvement. *Discrete Mathematics*, 4(4):367–377, 1973.

Kalai, G. A subexponential randomized simplex algorithm. In *Proceedings of the twenty-fourth annual ACM symposium on Theory of computing*, pp. 475–482, 1992.

Klee, V. and Minty, G. J. How good is the simplex algorithm. *Inequalities*, 3(3):159–175, 1972.

Lemke, C. E. The dual method of solving the linear programming problem. *Naval Research Logistics Quarterly*, 1(1):36–47, 1954.

Leyton-Brown, K., Nudelman, E., and Shoham, Y. Learning the empirical hardness of optimization problems: The case of combinatorial auctions. In Van Hentenryck, P. (ed.), *Principles and Practice of Constraint Programming - CP 2002*, pp. 556–572, 2002.

Matoušek, J. and Szabó, T. Random edge can be exponential on abstract cubes. *Advances in Mathematics*, 204(1):262–277, 2006.

NETLIB. Netlib database, 2010. data retrieved from http://www.netlib.org/lp/data/.

Paris, Q. *An economic interpretation of linear programming*. Springer, 2016.

Ploskas, N. Pivoting rules for the revised simplex algorithm. *Yugoslav Journal of Operations Research*, 24(3), 2016.

Read, C. B. and Belsley, D. A. Conditioning diagnostics: Collinearity and weak data in regression. 1991.

Schrijver, A. *Theory of linear and integer programming*. Wiley, 1989.

Shamir, R. The efficiency of the simplex method: A survey. *Management Science*, 33(3):301–334, 1987. URL http://www.jstor.org/stable/2631853.

Smale, S. On the average number of steps of the simplex method of linear programming. *Mathematical programming*, 27(3):241–262, 1983.

Terlaky, T. and Zhang, S. Pivot rules for linear programming: A survey on recent theoretical developments. *Annals of Operations Research*, 46(1):203–233, 1993.

Wieringen, W. V. Lecture notes on ridge regression, 2020.

Xu, L., Hutter, F., Hoos, H. H., and Leyton-Brown, K. Satzilla: portfolio-based algorithm selection for sat. *Journal of Artificial Intelligence Research*, 32:565–606, 2008.